

LAB REPORT

High Performance and Cloud Computing

Course Code : 23AID304

Submitted by

Kundurthi Abhiram

DL.AI.U4AID24144

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY (B.Tech)

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE (AIDS)

Semester V

Course Professor : Prof. Suraj Singh



AMRITA VISHWA VIDYAPEETHAM

School of Artificial Intelligence

Faridabad, Delhi NCR

September 2026

INDEX

EXP	Title / Description	Page No.
EXPERIMENT 1 – Shared and Private Clauses		
1.1	AIM	2
1.2	THEORY	2
1.3	CODE	3
1.4	OUTPUT	4
1.5	CONCLUSION	4
EXPERIMENT 2 – Firstprivate, Lastprivate and Threadprivate Clauses		
2.1	AIM	5
2.2	THEORY	5
2.3	FIRSTPRIVATE – CODE AND OUTPUT	6
2.4	LASTPRIVATE – CODE AND OUTPUT	7
2.5	THREADPRIVATE – CODE AND OUTPUT	8
2.6	CONCLUSION	9

Experiment 1: Shared and Private Clauses

AIM

To study and demonstrate the `shared` and `private` data-sharing clauses in OpenMP using C, and to observe how variables are accessed differently by multiple threads during parallel execution.

THEORY

OpenMP is an API for shared-memory parallel programming in C, C++ and Fortran. It allows a program to create multiple threads and execute a portion of the program concurrently. Data-sharing clauses determine whether variables are accessible through a common memory location or whether each thread receives an independent copy.

Shared Clause: When a variable is declared `shared`, all threads in the parallel region refer to the same memory location. Therefore, a modification made by one thread can be visible to other threads, subject to the normal synchronization and data-race rules of OpenMP.

Private Clause: When a variable is declared `private`, every thread receives its own independent copy of the variable. The private copies are local to the threads and are not copied back to the original variable when the parallel region ends. A basic `private` variable should be assigned a value before its value is used inside the parallel region.

In this experiment, `shared_var` is declared `shared` and `private_var` is declared `private`. The thread number is used to assign different values to the private copies so that the difference can be observed directly.

CODE

```
#include <stdio.h>
#include <omp.h>

int main()
{
    int shared_var = 10;
    int private_var = 20;

    #pragma omp parallel shared(shared_var) private(private_var)
    {
        private_var = 100 + omp_get_thread_num();

        printf("Thread %d: shared = %d, private = %d\n",
               omp_get_thread_num(),
```

```

        shared_var,
        private_var);
    }

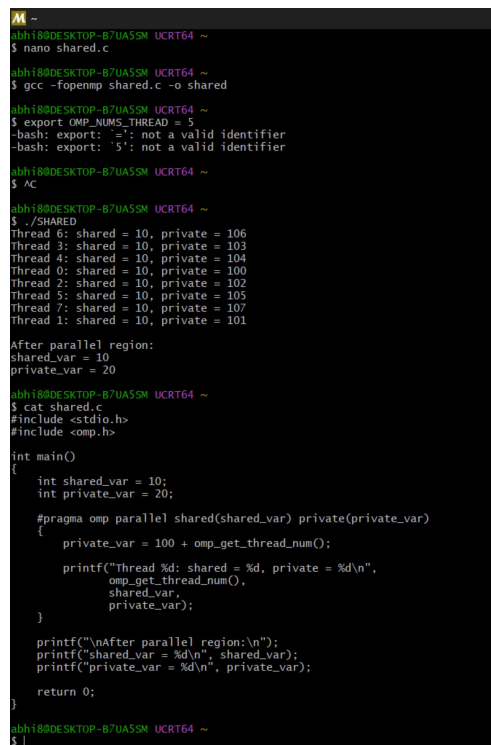
    printf("\nAfter parallel region:\n");
    printf("shared_var = %d\n", shared_var);
    printf("private_var = %d\n", private_var);

    return 0;
}

```

OUTPUT

The output shows that every thread reads the same value of `shared_var`, while each thread has a different value of `private_var`. The original value of `private_var` remains unchanged after the parallel region.



```

abhi188DESKTOP-B7UA5SM UCRT64 ~
$ nano shared.c
abhi188DESKTOP-B7UA5SM UCRT64 ~
$ gcc -fopenmp shared.c -o shared
abhi188DESKTOP-B7UA5SM UCRT64 ~
$ export OMP_NUMS_THREAD = 8
-bash: export: '=': not a valid identifier
-bash: export: '8': not a valid identifier
abhi188DESKTOP-B7UA5SM UCRT64 ~
$ ./shared
Thread 6: shared = 10, private = 106
Thread 3: shared = 10, private = 103
Thread 4: shared = 10, private = 104
Thread 0: shared = 10, private = 100
Thread 2: shared = 10, private = 102
Thread 5: shared = 10, private = 105
Thread 7: shared = 10, private = 107
Thread 1: shared = 10, private = 101
After parallel region:
shared_var = 10
private_var = 20
abhi188DESKTOP-B7UA5SM UCRT64 ~
$ cat shared.c
#include <stdio.h>
#include <omp.h>

int main()
{
    int shared_var = 10;
    int private_var = 20;

    #pragma omp parallel shared(shared_var) private(private_var)
    {
        private_var = 100 + omp_get_thread_num();

        printf("Thread %d: shared = %d, private = %d\n",
            omp_get_thread_num(),
            shared_var,
            private_var);
    }

    printf("\nAfter parallel region:\n");
    printf("shared_var = %d\n", shared_var);
    printf("private_var = %d\n", private_var);

    return 0;
}

```

Figure 1: Experiment 1: Shared and Private Clauses

CONCLUSION

The `shared` and `private` clauses were successfully demonstrated using OpenMP. The experiment verified that a shared variable is common to all threads, whereas a private variable has an independent copy for every thread. The original private variable also retained its value after the parallel region, illustrating the data-sharing behavior of OpenMP.

Experiment 2: Firstprivate, Lastprivate and Threadprivate Clauses

AIM

To study and implement the OpenMP `firstprivate`, `lastprivate`, and `threadprivate` clauses in C, and to understand how values are initialized, retained, and associated with individual threads during parallel execution.

THEORY

The `firstprivate` clause provides each thread with a private copy of a variable and initializes that copy with the value of the variable before entering the parallel construct. Changes made to these copies do not modify the original variable.

The `lastprivate` clause provides private copies to the threads executing a loop or other applicable construct and, at the end of the construct, copies the value associated with the logically last iteration back to the original variable. It is particularly useful with parallel loops when the value produced by the final sequential iteration is required after the loop.

The `threadprivate` directive applies to file-scope variables with static storage duration and gives each thread its own copy of the variable. Unlike an ordinary local `private` variable, the thread-specific copy can persist across separate parallel regions subject to the OpenMP rules governing `threadprivate` variables.

Thus, the three clauses can be remembered as follows:

- `firstprivate`: private copy with the original value copied **in**.
- `lastprivate`: private copy with the value from the logically last iteration copied **out**.
- `threadprivate`: a separate thread-specific copy of a global/static variable.

2.3 FIRSTPRIVATE

CODE

```
#include <stdio.h>
#include <omp.h>

int main()
{
    int x = 10;

    #pragma omp parallel firstprivate(x)
    {
        x = x + omp_get_thread_num();

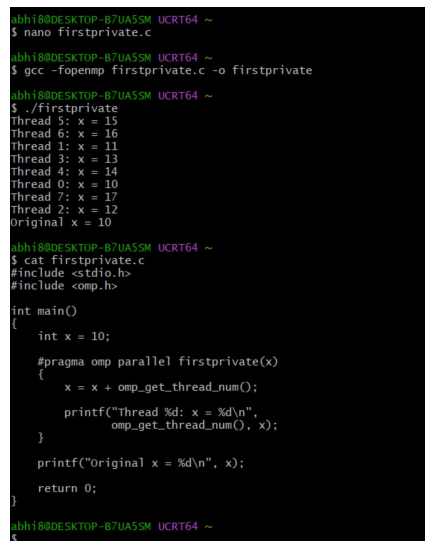
        printf("Thread %d: x = %d\n",
               omp_get_thread_num(), x);
    }

    printf("Original x = %d\n", x);

    return 0;
}
```

Each thread starts with its own copy of `x`, initialized to the original value 10. The thread number is then added to its private copy. Since the original variable is not modified by the threads, its value remains 10 after the parallel region.

OUTPUT



```
abhi@DESKTOP-B7UA5SM UCR164 ~
$ nano firstprivate.c
abhi@DESKTOP-B7UA5SM UCR164 ~
$ gcc -fopenmp firstprivate.c -o firstprivate
abhi@DESKTOP-B7UA5SM UCR164 ~
$ ./firstprivate
Thread 5: x = 15
Thread 6: x = 16
Thread 1: x = 11
Thread 3: x = 13
Thread 4: x = 14
Thread 0: x = 10
Thread 7: x = 17
Thread 2: x = 12
Original x = 10
abhi@DESKTOP-B7UA5SM UCR164 ~
$ cat firstprivate.c
#include <stdio.h>
#include <omp.h>

int main()
{
    int x = 10;

    #pragma omp parallel firstprivate(x)
    {
        x = x + omp_get_thread_num();

        printf("Thread %d: x = %d\n",
               omp_get_thread_num(), x);
    }

    printf("Original x = %d\n", x);

    return 0;
}
abhi@DESKTOP-B7UA5SM UCR164 ~
$
```

Figure 2: Output of the Firstprivate Clause

2.4 LASTPRIVATE

CODE

```
#include <stdio.h>
#include <omp.h>

int main()
{
    int x = 0;

    #pragma omp parallel for lastprivate(x)
    for (int i = 0; i < 5; i++)
    {
        x = i * 10;

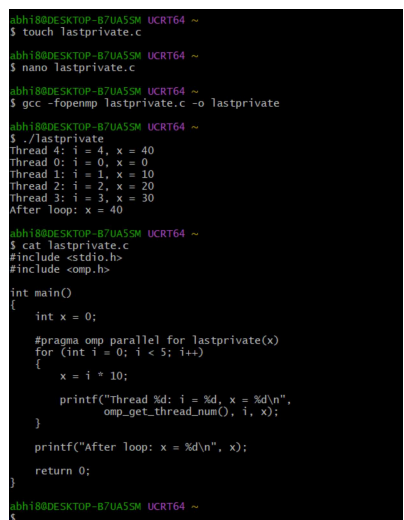
        printf("Thread %d: i = %d, x = %d\n",
               omp_get_thread_num(), i, x);
    }

    printf("After loop: x = %d\n", x);

    return 0;
}
```

The loop executes five iterations, from $i = 0$ to $i = 4$. Each iteration can be executed by a different thread. The logically last iteration is $i = 4$, for which $x = 40$. Therefore, after the parallel loop, the original variable contains the value 40.

OUTPUT



```
bbhi88DESKTOP-87UAS5M UCRT64 ~
$ touch lastprivate.c
bbhi88DESKTOP-87UAS5M UCRT64 ~
$ nano lastprivate.c
bbhi88DESKTOP-87UAS5M UCRT64 ~
$ gcc -fopenmp lastprivate.c -o lastprivate
bbhi88DESKTOP-87UAS5M UCRT64 ~
$ ./lastprivate
Thread 4: i = 4, x = 40
Thread 0: i = 0, x = 0
Thread 1: i = 1, x = 10
Thread 2: i = 2, x = 20
Thread 3: i = 3, x = 30
After loop: x = 40
bbhi88DESKTOP-87UAS5M UCRT64 ~
$ cat lastprivate.c
#include <stdio.h>
#include <omp.h>

int main()
{
    int x = 0;

    #pragma omp parallel for lastprivate(x)
    for (int i = 0; i < 5; i++)
    {
        x = i * 10;

        printf("Thread %d: i = %d, x = %d\n",
               omp_get_thread_num(), i, x);
    }

    printf("After loop: x = %d\n", x);

    return 0;
}
```

Figure 3: Output of the Lastprivate Clause

2.5 THREADPRIVATE

CODE

```
#include <stdio.h>
#include <omp.h>

int x = 10;

#pragma omp threadprivate(x)

int main()
{
    #pragma omp parallel
    {
        x = 100 + omp_get_thread_num();

        printf("Thread %d: x = %d\n",
               omp_get_thread_num(), x);
    }

    return 0;
}
```

The variable `x` is declared at file scope and is made `threadprivate` using the OpenMP directive. Consequently, each thread accesses its own copy of `x`. The assignment based on `omp_get_thread_num()` demonstrates that changes to one thread's copy do not affect the copy belonging to another thread.

OUTPUT

```
abhi8@DESKTOP-B7UA5SM UCRT64 ~  
$ touch threadprivate.c  
  
abhi8@DESKTOP-B7UA5SM UCRT64 ~  
$ nano threadprivate.c  
  
abhi8@DESKTOP-B7UA5SM UCRT64 ~  
$ gcc -fopenmp threadprivate.c -o threadprivate  
  
abhi8@DESKTOP-B7UA5SM UCRT64 ~  
$ ./threadprivate  
Thread 2: x = 102  
Thread 6: x = 106  
Thread 4: x = 104  
Thread 7: x = 107  
Thread 5: x = 105  
Thread 3: x = 103  
Thread 0: x = 100  
Thread 1: x = 101  
  
abhi8@DESKTOP-B7UA5SM UCRT64 ~  
$ cat threadprivate.c  
#include <stdio.h>  
#include <omp.h>  
  
int x = 10;  
  
#pragma omp threadprivate(x)  
  
int main()  
{  
    #pragma omp parallel  
    {  
        x = 100 + omp_get_thread_num();  
  
        printf("Thread %d: x = %d\n",  
              omp_get_thread_num(), x);  
    }  
  
    return 0;  
}
```

Figure 4: Output of the Threadprivate Clause

2.6 CONCLUSION

The `firstprivate`, `lastprivate`, and `threadprivate` OpenMP constructs were successfully implemented and studied. `firstprivate` demonstrated initialization of a separate copy for every thread, `lastprivate` demonstrated copying the value from the logically last loop iteration back to the original variable, and `threadprivate` demonstrated separate thread-specific copies of a global variable. These experiments provided practical understanding of OpenMP data-sharing attributes and their role in shared-memory parallel programming.